

```
1 import java.util.*;
2
3 public class BFS {
4     // This method will perform BFS on the graph starting from the given start node
5     public static void bfs(Map<Integer, List<Integer>> graph, int start) {
6         // Queue to manage nodes to visit
7         Queue<Integer> queue = new LinkedList<>();
8
9         // Set to keep track of visited nodes
10        Set<Integer> visited = new HashSet<>();
11
12        // Start with the start node
13        queue.add(start);
14        visited.add(start);
15
16        // Process the queue
17        while (!queue.isEmpty()) {
18            // Get the current node
19            int currentNode = queue.poll();
20            System.out.print(currentNode + " "); // Process the current node
21
22            // Get all the neighbors of the current node
23            List<Integer> neighbors = graph.get(currentNode);
24
25            // Explore all unvisited neighbors
26            for (int neighbor : neighbors) {
27                if (!visited.contains(neighbor)) {
28                    visited.add(neighbor);
29                    queue.add(neighbor); // Add to the queue for future processing
30                }
31            }
32        }
33    }
34
35    public static void main(String[] args) {
36        // Create a graph using an adjacency list representation
37        Map<Integer, List<Integer>> graph = new HashMap<>();
38
39        graph.put(1, Arrays.asList(2, 3, 4));
40        graph.put(2, Arrays.asList(1));
41        graph.put(3, Arrays.asList(1, 6));
42        graph.put(4, Arrays.asList(1, 5));
43        graph.put(5, Arrays.asList(4, 6, 7));
44        graph.put(6, Arrays.asList(3, 5, 7));
45        graph.put(7, Arrays.asList(5, 7));
46
47        // Perform BFS starting from node 0
48        System.out.println("BFS Traversal starting from node 1:");
49        bfs(graph, 1);
50    }
51 }
52
```

```
1 import java.util.*;
2
3 public class DFS {
4     // This method will perform DFS on the graph starting from the given start node using a stack
5     // It always explores the smallest unvisited neighbor first
6     public static void dfs(Map<Integer, List<Integer>> graph, int start) {
7         // Stack to manage nodes to visit
8         Stack<Integer> stack = new Stack<>();
9
10        // Set to keep track of visited nodes
11        Set<Integer> visited = new HashSet<>();
12
13        // Start with the start node
14        stack.push(start);
15
16        while (!stack.isEmpty()) {
17            // Pop the top node from the stack
18            int currentNode = stack.pop();
19
20            // If the node hasn't been visited yet
21            if (!visited.contains(currentNode)) {
22                visited.add(currentNode);
23                System.out.print(currentNode + " "); // Process the current node
24
25                // Get all the neighbors of the current node
26                List<Integer> neighbors = graph.get(currentNode);
27
28                // Sort the neighbors to always visit the smallest one first
29                Collections.sort(neighbors);
30
31                // Push all unvisited neighbors onto the stack (in reverse order)
32                for (int i = neighbors.size() - 1; i >= 0; i--) {
33                    int neighbor = neighbors.get(i);
34                    if (!visited.contains(neighbor)) {
35                        stack.push(neighbor);
36                    }
37                }
38            }
39        }
40    }
41
42    public static void main(String[] args) {
43        // Create a graph using an adjacency list representation
44        Map<Integer, List<Integer>> graph = new HashMap<>();
45
46        graph.put(1, Arrays.asList(2, 3, 4));
47        graph.put(2, Arrays.asList(1));
48        graph.put(3, Arrays.asList(1, 6));
49        graph.put(4, Arrays.asList(1, 5));
50        graph.put(5, Arrays.asList(4, 6, 7));
51        graph.put(6, Arrays.asList(3, 5, 7));
52        graph.put(7, Arrays.asList(5, 7));
53
54        // Perform DFS starting from node 1
55        System.out.println("DFS Traversal starting from node 1:");
56        dfs(graph, 1);
57    }
58 }
59
```

```
1 import java.util.Random;
2
3 public class City {
4     private double temp;
5     private String name;
6
7     Random rand = new Random();
8
9     public City(String name, double temp) {
10         this.name = name;
11         this.temp = temp;
12     }
13
14     public double getTemp() {
15         return temp;
16     }
17
18     public String getName() {
19         return name;
20     }
21
22     public String stringExtensionCity() {
23         return this.name + ": ";
24     }
25 }
26
```

```

1 import java.io.*;
2 import java.util.ArrayList;
3 import java.util.Collections;
4 import java.util.HashMap;
5 import java.util.StringJoiner;
6
7 public class Rows {
8
9     HashMap<String, ArrayList<Double>> data = new HashMap<>();
10    HashMap<String, ArrayList<Double>> dataAverage = new HashMap<>();
11
12
13    public void gatherInformation() throws Exception {
14        BufferedReader bufferedReader = new BufferedReader(new FileReader("maturita/rows/data.txt"));
15
16        while (bufferedReader.ready()) {
17            String line = bufferedReader.readLine();
18            String[] lineData = line.split(";");
19            String city = lineData[0];
20            String temperature = lineData[1].replace(",", ".");
21
22            data.putIfAbsent(city, new ArrayList<>());
23            data.get(city).add(Double.parseDouble(temperature));
24        }
25
26        bufferedReader.close();
27    }
28
29    public void printInformation() {
30        for (String city : data.keySet()) {
31            System.out.println(city + "; " + data.get(city));
32        }
33    }
34
35    public void createFinal() {
36        for (String city : data.keySet()) {
37            dataAverage.putIfAbsent(city, new ArrayList<>());
38            dataAverage.get(city).add(getMinimumTemperature(data.get(city)));
39            dataAverage.get(city).add(getAverageTemperature(data.get(city)));
40            dataAverage.get(city).add(getMaximumTemperature(data.get(city)));
41        }
42    }
43
44    public void printFinal() {
45        StringJoiner stringJoiner = new StringJoiner(", ");
46
47        for (String city : dataAverage.keySet()) {
48            stringJoiner.add(city + "=" + dataAverage.get(city).get(0) + "/" + dataAverage.get(city).get(1)
49                + "/" + dataAverage.get(city).get(2));
50        }
51
52        System.out.println(stringJoiner.toString());
53    }
54
55    private double getAverageTemperature(ArrayList<Double> temps) {
56        Double sum = 0.0;
57
58        for (Double temp : temps) {
59            sum += temp;
60        }
61
62        return Math.round((sum / temps.size()) * 10.0) / 10.0 ;
63    }
64
65    private double getMinimumTemperature(ArrayList<Double> temps) {
66        return Collections.min(temps);
67    }
68
69    private double getMaximumTemperature(ArrayList<Double> temps) {
70        return Collections.max(temps);
71    }
72
73
74

```

```
75     public static void main(String[] args) throws Exception {
76
77         Rows demo = new Rows();
78
79         demo.gatherInformation();
80         demo.printInformation();
81
82         demo.createFinal();
83         demo.printFinal();
84     }
85 }
86
```

```
1 package backTrackAlgorithms;
2
3 public class Cross {
4     private final int[] cr;
5     private int level;
6
7     public Cross() {
8         this.cr = new int[5];
9         level = -1;
10        backTrack();
11    }
12
13    public int[] getCr() {
14        return cr;
15    }
16
17    public void backTrack() {
18        level++;
19
20        for (int i = 1; i <= 8; i++) {
21            cr[level] = i;
22            if (test()) {
23                backTrack();
24            }
25        }
26
27        level--;
28    }
29
30    public boolean test(){
31        return switch (level) {
32            case 1 -> cr[0] + cr[1] == 10;
33            case 2 -> cr[0] > cr[2];
34            case 3 -> cr[0] - cr[3] > 2;
35            case 4 -> {
36                if (cr[0] * cr[4] > 20) {
37                    System.out.printf(" %d \n", cr[1]);
38                    System.out.printf("%d %d %d\n", cr[2], cr[0], cr[3]);
39                    System.out.printf(" %d \n", cr[4]);
40                    System.out.println();
41                }
42                yield false;
43            }
44            default -> true;
45        };
46    }
47
48
49    public int getLevel() {
50        return level;
51    }
52
53    public void setLevel(int level) {
54        this.level = level;
55    }
56 }
57
```

```
1 public class GCD_LCM {
2     private final long num1;
3     private final long num2;
4
5     public GCD_LCM(long num1, long num2) {
6         if (num1 == num2){
7             this.num1 = num1;
8             this.num2 = num2;
9         }
10        else {
11            this.num1 = Math.max(num1, num2);
12            this.num2 = Math.min(num1, num2);
13        }
14    }
15
16    public long getNum1() {
17        return num1;
18    }
19
20    public long getNum2() {
21        return num2;
22    }
23
24    private long GCD(long num1, long num2) {
25        if (num2 != 0) {
26            return GCD(num2, num1 % num2);
27        }
28        else {
29            return num1;
30        }
31    }
32
33    public long getGCD(){
34        return GCD(num1,num2);
35    }
36
37    public long getLCM(){
38        return (num1*num2)/(GCD(num1,num2));
39    }
40
41    public static void main(String[] args) {
42        GCD_LCM gcdLcm = new GCD_LCM(4, 7);
43        System.out.printf("GCD(%d, %d) = %d\n", gcdLcm.getNum1(), gcdLcm.getNum2(), gcdLcm.getGCD());
44        System.out.printf("LCM(%d, %d) = %d\n", gcdLcm.getNum1(), gcdLcm.getNum2(), gcdLcm.getLCM());
45    }
46 }
47
```

```
1 import java.io.BufferedWriter;
2 import java.io.FileWriter;
3 import java.io.IOException;
4 import java.util.ArrayList;
5 import java.util.Random;
6
7 public class RowsGen {
8
9     BufferedWriter bufferedWriter;
10
11     ArrayList<City> cities;
12
13     Random rand = new Random();
14
15     int numOfLines;
16
17     public RowsGen(String filePath, ArrayList<City> cities, int numOfLines) throws IOException {
18         bufferedWriter = new BufferedWriter(new FileWriter(filePath, false));
19         this.cities = cities;
20         this.numOfLines = numOfLines;
21
22         long time = System.currentTimeMillis();
23         generateText();
24         System.out.printf("Time taken: %d", System.currentTimeMillis() - time);
25     }
26
27     public void generateText() throws IOException {
28         for (int i = 0; i < numOfLines; i++) {
29             writeText(getRandomCity(), getRandTempDifference());
30         }
31
32         bufferedWriter.close();
33     }
34
35     public void writeText(City city, double tempDif) throws IOException {
36         bufferedWriter.write(String.format("%s%.1f", city.stringExtensionCity(), city.getTemp() + tempDif));
37         bufferedWriter.newLine();
38     }
39
40     public City getRandomCity() {
41         return cities.get(rand.nextInt(cities.size()));
42     }
43
44     public double getRandTempDifference() {
45         double temp = rand.nextDouble(40) - 20;
46         return temp;
47     }
48 }
49
50
```



```
1 public class DecToBin {
2
3     public static String decToBin_classic(int cislo) {
4         String result = "";
5         while(cislo > 0) {
6             int podil = cislo / 2;
7             int zbytek = cislo % 2;
8             result += zbytek;
9             cislo = podil;
10        }
11        //vysledek je ve stringu opacne
12        //prevraceni stringu - oldschool
13        /*
14        String result_opacne = "";
15        for(int i = result.length() - 1; i >= 0; i--) {
16            result_opacne += result.charAt(i);
17        }
18        */
19        StringBuilder result_sb = new StringBuilder(result);
20        StringBuilder result_opacne = result_sb.reverse();
21        return result_opacne.toString();
22    }
23
24    public static String decToBin_rek(int cislo) {
25        int zbytek;
26        if(cislo <= 1) {
27            //nerekurzivni vetev
28            return Integer.toString(cislo);
29        } else {
30            //rekurzivni
31            zbytek = cislo % 2;
32            return decToBin_rek(cislo / 2) + zbytek;
33        }
34    }
35
36    public static void main(String[] args) {
37        System.out.println(decToBin_classic(11));
38        System.out.println(decToBin_rek(11));
39
40        for(int i = 0; i <= 100; i++) {
41            System.out.println(i + " - " + decToBin_rek(i));
42        }
43    }
44 }
45
```

```
1 <?php
2     function isLoggedIn() {
3         if (isset($_SESSION[session_id()])) {
4             return true;
5         }
6         else {
7             return false;
8         }
9     }
10
11     function logOut() {
12         if (isLoggedIn()) {
13             unset($_SESSION[session_id()]);
14             return true;
15         }
16         else {
17             return false;
18         }
19     }
20
21     function login($username, $password, $db) {
22         $password = md5($password);
23         $params = array($username, $password);
24
25
26         try {
27             $query = $db->prepare("SELECT COUNT(*) FROM uziv WHERE uziv_login = ? AND uziv_heslo = ?");
28             $query->execute($params);
29         }
30         catch (PDOException $e) {
31             die("Database error: " . $e->getMessage());
32         }
33
34         if ($query->fetchColumn(0) == 1) {
35             session_regenerate_id();
36             try {
37                 $query = $db->prepare("SELECT uziv_id FROM uziv WHERE uziv_login = ? AND uziv_heslo = ?");
38                 $query->execute($params);
39             }
40             catch (PDOException $e) {
41                 die("Database error: " . $e->getMessage());
42             }
43             $id = $query->fetchColumn(0);
44             $_SESSION[session_id()] = $id;
45             return true;
46         }
47         else {
48             return false;
49         }
50     }
51 ?>
52
```

```
1 import java.util.Arrays;
2 import java.util.Random;
3
4 public class Matrices {
5
6     static Random random = new Random();
7
8     static int[][] generateMatrix(int row, int column) {
9         int[][] result = new int[row][column];
10
11         for (int i = 0; i < result.length; i++) {
12             for (int j = 0; j < result[i].length; j++) {
13                 result[i][j] = random.nextInt(10);
14             }
15         }
16
17         return result;
18     }
19
20     static int[][] addMatrices(int[][] m1, int[][] m2) {
21         if (m1.length == m2.length && m1[0].length == m2[0].length) {
22             int[][] result = new int[m1.length][m1[0].length];
23
24             for (int i = 0; i < result.length; i++) {
25                 for (int j = 0; j < result[i].length; j++) {
26                     result[i][j] = m1[i][j] + m2[i][j];
27                 }
28             }
29
30             return result;
31         }
32         else {
33             throw new IllegalArgumentException("Matrices are not the same size");
34         }
35     }
36
37     static int[][] transformMatrix(int[][] m) {
38         int[][] result = new int[m[0].length][m.length];
39
40         for (int i = 0; i < result.length; i++) {
41             for (int j = 0; j < result[i].length; j++) {
42                 result[i][j] = m[j][i];
43             }
44         }
45
46         return result;
47     }
48
49     static int[][] multiplyMatrices(int[][] m1, int[][] m2) {
50
51         int rowsM1 = m1.length;
52         int rowsM2 = m2.length;
53         int columnsM1 = m1[0].length;
54         int columnsM2 = m2[0].length;
55
56         if (columnsM1 == rowsM2) {
57             int[][] result = new int[rowsM1][columnsM2];
58
59             for (int i = 0; i < rowsM1; i++) {
60                 for (int j = 0; j < columnsM2; j++) {
61                     for (int k = 0; k < columnsM1; k++) {
62                         result[i][j] += m1[i][k] * m2[k][j];
63                     }
64                 }
65             }
66
67             return result;
68         }
69         else {
70             throw new IllegalArgumentException(
71                 "Amount of columns in matrix 1 does not match the amount of rows in matrix 2");
72         }
73     }
74 }
```

```
75     static void printMatrix(int[][] m) {
76         String[][] result = new String[m.length][m[0].length];
77
78         for (int i = 0; i < result.length; i++) {
79             for (int j = 0; j < result[i].length; j++) {
80                 result[i][j] = String.format("%2d", m[i][j]);
81             }
82         }
83
84         for (String[] row : result) {
85             System.out.println(Arrays.toString(row));
86         }
87     }
88
89
90     public static void main(String[] args) {
91         int[][] m1 = generateMatrix(5, 3);
92         int[][] m2 = generateMatrix(3, 4);
93
94         System.out.println("Matrix 1");
95         printMatrix(m1);
96         System.out.println("\nMatrix 2");
97         printMatrix(m2);
98
99         // System.out.println("\nMatrix addition");
100        // printMatrix(addMatrices(m1, m2));
101
102        System.out.println("\nMatrix 1 transformation");
103        printMatrix(transformMatrix(m1));
104
105        System.out.println("\nMatrix multiplication");
106        printMatrix(multiplyMatrices(m1, m2));
107    }
108 }
```

```

1 import java.util.*;
2
3 public class AlgoritmyTest {
4
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7         Random rnd = new Random();
8
9         System.out.println("Jak velké pole chcete vygenerovat?");
10        int[] pole = new int[sc.nextInt()];
11        System.out.println("Pole bude doplněno náhodnými čísly");
12        System.out.println("Zadejte nejmenší generovatelné číslo:");
13        int min = sc.nextInt();
14        System.out.println("Zadejte největší generovatelné číslo:");
15        int max = sc.nextInt();
16
17        for (int i = 0; i < pole.length; i++) {
18            pole[i] = min + rnd.nextInt(max - min + 1);
19        }
20
21        System.out.println();
22        prvniVolba();
23        int volba = sc.nextInt();
24
25        switch (volba) {
26            case 1:
27                volbaRazeni();
28                volba = sc.nextInt();
29                switch (volba) {
30                    case 1:
31                        vypsaniPole(pole);
32                        bubbleSort(pole);
33                    break;
34                    case 2:
35                        vypsaniPole(pole);
36                        insertSort(pole);
37                    break;
38                    case 3:
39                        vypsaniPole(pole);
40                        selectSort(pole);
41                    break;
42                    case 4:
43                        vypsaniPole(pole);
44                        System.out.println();
45                        int[] finalpole = quickSort(0, pole.length-1, pole);
46                        System.out.println("Seřazené pole:");
47                        for (int i = 0; i < finalpole.length; i++) {
48                            System.out.format("%6d", finalpole[i]);
49                        }
50                    break;
51                    default:
52                        System.out.println("Neplatná volba");
53                        System.exit(0);
54                    break;
55                }
56            break;
57            case 2:
58                System.out.println("Jaký prvek chcete v poli vyhledávat?");
59                int x = sc.nextInt();
60                volbaVyhledavani();
61                volba = sc.nextInt();
62                switch (volba) {
63                    case 1:
64                        vypsaniPole(pole);
65                        naivniVyhledavani(pole, x);
66                    break;
67                    case 2:
68                        vypsaniPole(pole);
69                        bezZarazky(pole, x);
70                    break;
71                    case 3:
72                        vypsaniPole(pole);
73                        seZarazkou(pole, x);
74                    break;

```

```

75         case 4:
76             vypsaniPole(pole);
77             int[] finalpole = quickSort(0, pole.length-1, pole);
78             int vysledek = binarniVyhledavani(finalpole, 0, finalpole.length-1, x);
79             if (vysledek == -1) {
80                 System.out.println("Prvek se v poli nenachází");
81             } else {
82                 System.out.println("Prvek se v poli nachází");
83             }
84             break;
85         default:
86             System.out.println("Neplatná volba");
87             System.exit(0);
88         break;
89     }
90     break;
91     default:
92         System.out.println("Neplatná volba");
93         System.exit(0);
94     break;
95 }
96 }
97
98 public static void prvniVolba() {
99     System.out.println("Jakou operaci chcete provést?");
100    System.out.println("1 - Seřadit pole");
101    System.out.println("2 - Vyhledat prvek v poli");
102 }
103
104 public static void volbaRazeni() {
105     System.out.println("Jaký algoritmus chcete použít?");
106     System.out.println("1 - Bubble Sort");
107     System.out.println("2 - Insert Sort");
108     System.out.println("3 - Select Sort");
109     System.out.println("4 - Quick Sort");
110 }
111
112 public static void volbaVyhledavani() {
113     System.out.println("Jaký způsob vyhledávání chcete použít?");
114     System.out.println("1 - Naivní");
115     System.out.println("2 - Bez zarážky");
116     System.out.println("3 - Se zarážkou");
117     System.out.println("4 - Binární");
118 }
119
120 public static void vypsaniPole (int[] pole) {
121     System.out.println("Vygenerované pole:");
122     for (int i = 0; i<pole.length; i++) {
123         System.out.format("%6d", pole[i]);
124     }
125     System.out.println();
126 }
127
128 public static void bubbleSort(int[] pole) {
129     for (int i = 0; i<(pole.length-1); i++) {
130         for (int j = 0; j<(pole.length-1-i); j++) {
131             if (pole[j] > pole[j+1]) {
132                 int pom = pole[j];
133                 pole[j] = pole[j+1];
134                 pole[j+1] = pom;
135             }
136         }
137     }
138
139     System.out.println("Seřazené pole:");
140     for (int i = 0; i<pole.length; i++) {
141         System.out.format("%6d", pole[i]);
142     }
143 }
144
145
146
147
148

```

```

149     public static void insertSort(int[] pole) {
150         for (int i = 1; i < pole.length; i++) {
151             int ins = pole[i];
152             int j = i-1;
153             while ((j >= 0) && (pole[j] > ins)) {
154                 pole[j+1] = pole[j];
155                 j--;
156             }
157             pole[j+1] = ins;
158         }
159
160         System.out.println("Seřazené pole:");
161         for (int i = 0; i < pole.length; i++) {
162             System.out.format("%6d", pole[i]);
163         }
164     }
165
166     public static void selectSort(int[] pole) {
167         for (int i = 0; i < pole.length-1; i++) {
168             int max_pos = pole.length-1-i;
169             for (int j = 0; j < pole.length-i; j++) {
170                 if (pole[j] > pole[max_pos]) {
171                     max_pos = j;
172                 }
173             }
174             int pom = pole[pole.length-1-i];
175             pole[pole.length-1-i] = pole[max_pos];
176             pole[max_pos] = pom;
177         }
178
179         System.out.println("Seřazené pole:");
180         for (int i = 0; i < pole.length; i++) {
181             System.out.format("%6d", pole[i]);
182         }
183     }
184
185     public static int[] quickSort(int l, int r, int[] pole) {
186         int i = l;
187         int j = r;
188         int pivot = pole[((l + r) / 2)];
189
190         do {
191             while (pole[i] < pivot) i++;
192             while (pole[j] > pivot) j--;
193             if (i <= j) {
194                 int pom = pole[i];
195                 pole[i] = pole[j];
196                 pole[j] = pom;
197                 i++;
198                 j--;
199             }
200         } while (i < j);
201
202         if ((j - l) > 0) quickSort(l, j, pole);
203         if ((r - i) > 0) quickSort(i, r, pole);
204
205         return pole;
206     }
207
208     public static void naivniVyhledavani(int[] p, int x) {
209         boolean vysledek = false;
210
211         for (int i = 0; i < p.length; i++) {
212             if (p[i] == x) {
213                 vysledek = true;
214             }
215         }
216
217         if (vysledek == true) {
218             System.out.println("Prvek se v poli nachází");
219         } else {
220             System.out.println("Prvek se v poli nenachází");
221         }
222     }

```

```
223
224     public static void bezZarazky(int[] p, int x) {
225         boolean vysledek = false;
226         int i = 0;
227
228         while (vysledek == false && i < p.length) {
229             if (p[i] == x) {
230                 vysledek = true;
231             }
232             i++;
233         }
234
235         if (vysledek == true) {
236             System.out.println("Prvek se v poli nachází");
237         } else {
238             System.out.println("Prvek se v poli nenachází");
239         }
240     }
241
242     public static void seZarazkou(int[] p, int x) {
243         int[] pole = new int[p.length+1];
244         for (int i = 0; i<p.length; i++) {
245             pole[i] = p[i];
246         }
247         pole[p.length] = x;
248
249         boolean vysledek = false;
250         int i = 0;
251
252         while (vysledek == false) {
253             if (pole[i] == x) {
254                 vysledek = true;
255             }
256             i++;
257         }
258
259         if (vysledek == true && i < pole.length) {
260             System.out.println("Prvek se v poli nachází");
261         } else {
262             System.out.println("Prvek se v poli nenachází");
263         }
264     }
265
266     public static int binarniVyhledavani(int[] pole, int l, int r, int x) {
267         if (r >= l) {
268             int s = l + (r - l) / 2;
269
270             if (pole[s] == x) {
271                 return s;
272             }
273             if (pole[s] > x) {
274                 return binarniVyhledavani(pole, l, s-1, x);
275             } else {
276                 return binarniVyhledavani(pole, s+1, r, x);
277             }
278         }
279         return -1;
280     }
281
282 }
```



```
1 public class ArabicToRoman {
2
3     public static String arabicToRoman(int arabic) {
4         //pole arabskych konstant
5         int arabicNums[] = {1000, 900, 500, 400, 100, 90, 50,
6             40, 10, 9, 5, 4, 1};
7         String romanNums[] = {"M", "CM", "D", "CD", "C", "XC", "L",
8             "XL", "X", "IX", "V", "IV", "I"};
9
10        //String s vysledkem
11        String result = "";
12
13        //projdi vsechny arabske konstanty
14        for(int i = 0; i < arabicNums.length; i++) {
15            //dokud se aktualni arabska cislice vejde do prevadeneho cisla
16            while(arabic >= arabicNums[i]) {
17                //do vysledku pricteme rimsky ekvivalent aktualni arabske cislice
18                result += romanNums[i];
19                //od prevadene cislice odeceme aktualni arabskou
20                arabic -= arabicNums[i];
21            }
22        }
23
24        //vratime vysledek
25        return result;
26    }
27
28    public static void main(String[] args) {
29        // TODO Auto-generated method stub
30        System.out.println(arabicToRoman(3894));
31
32        for(int i = 0; i <= 3000; i++) {
33            System.out.println(i + " - " + arabicToRoman(i));
34        }
35    }
36 }
```

```
1 public class RomanToArabic {
2
3     public static int romanToArabic(String roman) {
4         //vysledek
5         int result = 0;
6         //aktualni hodnota - prave ji ctu
7         int cur = 0;
8         //predchozi hodnota - cetl jsem ji minule
9         int prev = 0;
10
11         //cteme rimske cislo odzadu
12         for(int i = roman.length() - 1; i >= 0; i--) {
13             //nacti znak
14             char znak = roman.charAt(i);
15             //nastav aktualni cislici
16             switch(znak) {
17                 case 'I':
18                     cur = 1;
19                     break;
20                 case 'V':
21                     cur = 5;
22                     break;
23                 case 'X':
24                     cur = 10;
25                     break;
26                 case 'L':
27                     cur = 50;
28                     break;
29                 case 'C':
30                     cur = 100;
31                     break;
32                 case 'D':
33                     cur = 500;
34                     break;
35                 case 'M':
36                     cur = 1000;
37                     break;
38             }
39
40             //porovnej jestli aktualni cislice je vyssi nebo rovna te minule
41             if(cur >= prev) {
42                 //ano - neni poruseno pravidlo od mensich k vetsim => pricitame
43                 result += cur;
44             } else {
45                 //ne - je poruseno pravidlo od mensich k vetsim => odcitame
46                 result -= cur;
47             }
48             //aktualizujeme minule cislo
49             prev = cur;
50         }
51
52         //vratime vysledek
53         return result;
54     }
55
56     public static void main(String[] args) {
57         // TODO Auto-generated method stub
58         System.out.println(romanToArabic("MMMCCCXCIV"));
59         System.out.println(romanToArabic(" "));
60     }
61 }
```

```
1 import java.util.Scanner;
2
3 public class BaseConversion {
4     public static void main(String[] args) {
5         Scanner scanner = new Scanner(System.in);
6
7         System.out.print("Enter Number: ");
8         String cislo = scanner.next();
9
10        System.out.print("From which base: ");
11        int fromBase = scanner.nextInt();
12
13        System.out.print("To which base: ");
14        int toBase = scanner.nextInt();
15
16        try {
17            int decimalValue = Integer.parseInt(cislo, fromBase);
18
19            String result = Integer.toString(decimalValue, toBase);
20
21            System.out.println("Result: " + result.toUpperCase());
22        } catch (NumberFormatException e) {
23            System.out.println("Invalid number or base");
24        }
25
26        scanner.close();
27    }
28 }
```

```

1 import java.net.URL;
2 import java.util.ResourceBundle;
3
4 import javafx.beans.value.ChangeListener;
5 import javafx.beans.value.ObservableValue;
6 import javafx.fxml.FXML;
7 import javafx.fxml.Initializable;
8 import javafx.geometry.Insets;
9 import javafx.scene.control.Slider;
10 import javafx.scene.layout.Pane;
11 import javafx.scene.layout.Background;
12 import javafx.scene.layout.BackgroundFill;
13 import javafx.scene.layout.CornerRadii;
14 import javafx.scene.paint.Color;
15
16 public class ColorMixerController implements Initializable {
17
18     //promenne pro pamatovani nastavenych barev
19     private int red = 0;
20     private int green = 0;
21     private int blue = 0;
22
23     //graf. komponenty z FXML
24     @FXML
25     private Pane pnl_color;
26     @FXML
27     private Slider sld_red;
28     @FXML
29     private Slider sld_green;
30     @FXML
31     private Slider sld_blue;
32
33     @Override
34     public void initialize(URL url, ResourceBundle rb) {
35         //tato metoda se spusti pri startu aplikace
36
37         //ozivime red slider
38         sld_red.valueProperty().addListener(new ChangeListener<Number>() {
39             @Override
40             public void changed(ObservableValue<? extends Number> observable,
41                 Number oldValue, Number newValue) {
42                 //sem vlozit kod, ktery se spusti pri pohnuti se sliderem
43                 //stahni novou hodnotu cerveneho slideru a uloz ji do promenne
44                 red = newValue.intValue();
45                 //prebarvi pozadi panelu
46                 setPanelBackGround();
47             }
48         });
49
50         //ozivime green slider
51         sld_green.valueProperty().addListener(new ChangeListener<Number>() {
52             @Override
53             public void changed(ObservableValue<? extends Number> observable,
54                 Number oldValue, Number newValue) {
55                 //sem vlozit kod, ktery se spusti pri pohnuti se sliderem
56                 //stahni novou hodnotu cerveneho slideru a uloz ji do promenne
57                 green = newValue.intValue();
58                 //prebarvi pozadi panelu
59                 setPanelBackGround();
60             }
61         });
62
63
64
65
66
67
68
69
70
71
72
73
74

```

```
75     //ozivime blue slider
76     sld_blue.valueProperty().addListener(new ChangeListener<Number>() {
77         @Override
78         public void changed(ObservableValue<? extends Number> observable,
79             Number oldValue, Number newValue) {
80             //sem vlozit kod, který se spusti pri pohnuti se sliderem
81             //stahni novou hodnotu cerveneho slideru a uloz ji do promenne
82             blue = newValue.intValue();
83             //prebarvi pozadi panelu
84             setPanelBackGround();
85         }
86     });
87
88     //spustime setBackGround - kvuli nastaveni prvni cerne barvy
89     setPanelBackGround();
90 }
91
92 private void setPanelBackGround() {
93     pnl_color.setBackground(new Background(
94         new BackgroundFill(Color.rgb(red, green, blue),
95             CornerRadii.EMPTY, Insets.EMPTY)));
96 }
97 }
```

```
1 import javafx.application.Application;
2 import javafx.stage.Stage;
3 import javafx.scene.Scene;
4 import javafx.scene.layout.AnchorPane;
5 import javafx.fxml.FXMLLoader;
6
7
8 public class ColorMixerApplication extends Application {
9     @Override
10    public void start(Stage primaryStage) {
11        try {
12            AnchorPane root = (AnchorPane)FXMLLoader.load(getClass().getResource("ColorMixerV2.fxml"));
13            Scene scene = new Scene(root);
14            primaryStage.setScene(scene);
15            primaryStage.show();
16        } catch (Exception e) {
17            e.printStackTrace();
18        }
19    }
20
21    public static void main(String[] args) {
22        launch(args);
23    }
24 }
```